

Mastering Stacks in Python: A Comprehensive Guide



Follow @techie_programmer for more Programming content!

Understanding Stacks: The Foundation

A **stack** is a fundamental linear data structure that operates on the **Last-In-First-Out (LIFO)** principle. Imagine a stack of plates in a cafeteria—you can only add new plates to the top, and when you need one, you take it from the top. The last plate placed is the first one removed.

Real-World Analogy

Think of these everyday examples:

- **Stack of books:** You add books on top and remove from the top
- **Browser back button:** Each page you visit is "pushed" onto a stack, and clicking back "pops" them off
- **Undo feature:** Each action is stacked, and undo removes the most recent action

Core Stack Operations

Every stack implementation supports five fundamental operations:

Operation	Description	Time Complexity
Push	Add an element to the top of the stack	O(1)
Pop	Remove and return the top element	O(1)
Peek/Top	View the top element without removing it	O(1)
isEmpty	Check if the stack contains any elements	O(1)
Size	Return the number of elements in the stack	O(1)

Visual Representation

Initial Stack: [5, 6, 2, 9]

↑
TOP

Push(8): [5, 6, 2, 9, 8] ← Element added here

↑
TOP

Pop(): [5, 6, 2, 9] ← 8 is removed

↑
TOP

Implementation Approach 1: Using Python Lists

Python lists provide built-in methods that make them ideal for quick stack implementations.

Quick Implementation

```

# Simple stack using Python list
stack = []

# Push operations
stack.append('A')
stack.append('B')
stack.append('C')
print("Stack after pushes:", stack)  # Output: ['A', 'B', 'C']

# Peek at top element
top_element = stack[-1]
print("Top element:", top_element)  # Output: C

# Pop operation
popped = stack.pop()
print("Popped element:", popped)  # Output: C
print("Stack after pop:", stack)  # Output: ['A', 'B']

# Check if empty
is_empty = len(stack) == 0
print("Is stack empty?", is_empty)  # Output: False

# Get size
print("Stack size:", len(stack))  # Output: 2

```

Professional Stack Class Implementation

For production code, a dedicated class provides better encapsulation, error handling, and maintainability:

```

class Stack:
    """A stack implementation using Python list."""

```

```

def __init__(self):
    """Initialize an empty stack."""
    self._items = []

def push(self, item):
    """
    Add an item to the top of the stack.

    Args:
        item: The element to be added
    """
    self._items.append(item)

def pop(self):
    """
    Remove and return the top item from the stack.

    Returns:
        The top element, or None if stack is empty

    Raises:
        IndexError: If the stack is empty
    """
    if self.is_empty():
        raise IndexError("Cannot pop from an empty stack")
    return self._items.pop()

def peek(self):
    """
    Return the top item without removing it.

    Returns:
        The top element, or None if stack is empty
    """
    if self.is_empty():
        raise IndexError("Cannot peek at an empty stack")
    return self._items[-1]

```

```

def is_empty(self):
    """Check if the stack is empty."""
    return len(self._items) == 0

def size(self):
    """Return the number of items in the stack."""
    return len(self._items)

def __str__(self):
    """String representation of the stack."""
    return f"Stack({self._items})"

def __repr__(self):
    """Official string representation."""
    return f"Stack({self._items})"

# Example usage
my_stack = Stack()

# Push multiple items
for item in ['A', 'B', 'C', 'D']:
    my_stack.push(item)
    print(f"Pushed {item}: {my_stack}")

print(f"\nStack size: {my_stack.size()}")
print(f"Top element: {my_stack.peek()}")

# Pop items
while not my_stack.is_empty():
    popped = my_stack.pop()
    print(f"Popped {popped}, remaining: {my_stack}")

```

Advantages of List-Based Implementation

- ✓ **Memory efficient:** No overhead for storing pointers/references
- ✓ **Simple and intuitive:** Easy to understand and implement
- ✓ **Fast operations:** Direct access to the end of the list
- ✓ **Built-in optimizations:** Python's list is highly optimized in C

Disadvantages of List-Based Implementation

- ✗ **Dynamic resizing overhead:** When capacity is exceeded, the entire list may need reallocation
- ✗ **Potential memory waste:** Pre-allocated space might go unused
- ✗ **Not truly unlimited:** Although dynamic, there's a practical memory limit

Implementation Approach 2: Using Linked Lists

A linked list implementation offers true dynamic sizing without reallocation overhead.

Node and Stack Classes

```
class Node:
    """A node in a linked list."""

    def __init__(self, value):
        """
        Initialize a node with a value.

        Args:
            value: The data to store in this node
        """
        self.value = value
        self.next = None

class LinkedListStack:
    """A stack implementation using a singly linked list."""
```

```

def __init__(self):
    """Initialize an empty stack."""
    self._top = None
    self._size = 0

def push(self, value):
    """
    Add a value to the top of the stack.

    Args:
        value: The element to be added
    """
    new_node = Node(value)
    new_node.next = self._top
    self._top = new_node
    self._size += 1

def pop(self):
    """
    Remove and return the top value from the stack.

    Returns:
        The top element

    Raises:
        IndexError: If the stack is empty
    """
    if self.is_empty():
        raise IndexError("Cannot pop from an empty stack")

    popped_value = self._top.value
    self._top = self._top.next
    self._size -= 1
    return popped_value

def peek(self):
    """

```

Return the top value without removing it.

Returns:

The top element

Raises:

IndexError: If the stack is empty

"""

```
if self.is_empty():
```

```
    raise IndexError("Cannot peek at an empty stack")
```

```
return self._top.value
```

```
def is_empty(self):
```

```
    """Check if the stack is empty."""
```

```
    return self._top is None
```

```
def size(self):
```

```
    """Return the number of items in the stack."""
```

```
    return self._size
```

```
def display(self):
```

```
    """Display all elements in the stack from top to  
bottom."""
```

```
    if self.is_empty():
```

```
        return "Stack is empty"
```

```
    elements = []
```

```
    current = self._top
```

```
    while current:
```

```
        elements.append(str(current.value))
```

```
        current = current.next
```

```
    return " -> ".join(elements) + " -> None"
```

```
def __str__(self):
```

```
    """String representation of the stack."""
```

```
    return f"LinkedListStack(top={self._top.value if  
self._top else None}, size={self._size})"
```

```

# Example usage
ll_stack = LinkedListStack()

# Push operations
print("=== Push Operations ===")
for item in ['X', 'Y', 'Z']:
    ll_stack.push(item)
    print(f"Pushed {item}: {ll_stack.display()}")

print(f"\nStack representation: {ll_stack}")
print(f"Stack size: {ll_stack.size()}")
print(f"Top element: {ll_stack.peek()}")

# Pop operations
print("\n=== Pop Operations ===")
while not ll_stack.is_empty():
    popped = ll_stack.pop()
    display = ll_stack.display() if not ll_stack.is_empty() else "Empty"
    print(f"Popped {popped}, remaining: {display}")

```

Advantages of Linked List Implementation

- ✓ **True dynamic sizing:** Grows and shrinks without reallocation
- ✓ **No wasted memory:** Only allocates what's needed
- ✓ **Predictable performance:** No occasional slowdowns from resizing
- ✓ **Memory fragmentation friendly:** Nodes can be scattered in memory

Disadvantages of Linked List Implementation

- ✗ **Extra memory per element:** Each node stores a pointer (8 bytes on 64-bit systems)
- ✗ **More complex code:** Requires node management and pointer handling

- ❌ **Cache unfriendly:** Non-contiguous memory reduces CPU cache efficiency
- ❌ **Slight overhead:** Additional allocations for each node

Comparison: List vs Linked List

Aspect	Python List	Linked List
Memory per element	Data only	Data + pointer
Memory efficiency	Better for small stacks	Better for large, dynamic stacks
Implementation complexity	Simple	Moderate
Cache performance	Excellent	Poor
Resize overhead	Occasional	None
Best for	Most use cases	Memory-constrained or highly dynamic scenarios

Real-World Applications

1. Expression Evaluation

```
def evaluate_postfix(expression):
    """Evaluate a postfix expression using a stack."""
    stack = Stack()

    for token in expression.split():
        if token.isdigit():
            stack.push(int(token))
        else:
            b = stack.pop()
```

```

        a = stack.pop()
        if token == '+':
            stack.push(a + b)
        elif token == '-':
            stack.push(a - b)
        elif token == '*':
            stack.push(a * b)
        elif token == '/':
            stack.push(a / b)

    return stack.pop()

# Example: "5 3 + 2 *" means (5 + 3) * 2 = 16
result = evaluate_postfix("5 3 + 2 *")
print(f"Result: {result}") # Output: 16

```

2. Balanced Parentheses Checker

```

def is_balanced(expression):
    """Check if parentheses in expression are balanced."""
    stack = Stack()
    matching = {'(': ')', '[': ']', '{': '}'}

    for char in expression:
        if char in matching: # Opening bracket
            stack.push(char)
        elif char in matching.values(): # Closing bracket
            if stack.is_empty():
                return False
            if matching[stack.pop()] != char:
                return False

    return stack.is_empty()

```

```
# Test cases
print(is_balanced("((a + b) * [c - d])")) # True
print(is_balanced("((a + b]"))           # False
```

3. Undo/Redo Functionality

```
class TextEditor:
    """Simple text editor with undo/redo using stacks."""

    def __init__(self):
        self.text = ""
        self.undo_stack = Stack()
        self.redo_stack = Stack()

    def write(self, new_text):
        """Add text and save state for undo."""
        self.undo_stack.push(self.text)
        self.text += new_text
        # Clear redo stack when new action is performed
        self.redo_stack = Stack()

    def undo(self):
        """Undo the last write operation."""
        if not self.undo_stack.is_empty():
            self.redo_stack.push(self.text)
            self.text = self.undo_stack.pop()

    def redo(self):
        """Redo the last undone operation."""
        if not self.redo_stack.is_empty():
            self.undo_stack.push(self.text)
            self.text = self.redo_stack.pop()

# Example usage
```

```
editor = TextEditor()
editor.write("Hello ")
editor.write("World")
print(f"Text: '{editor.text}'") # Hello World

editor.undo()
print(f"After undo: '{editor.text}'") # Hello

editor.redo()
print(f"After redo: '{editor.text}'") # Hello World
```

Stack Applications Summary

- ◆ **Function Call Stack:** Programming languages use stacks to manage function calls and local variables
- ◆ **Browser History:** Back/forward navigation in web browsers
- ◆ **Undo/Redo:** Text editors, graphic design software
- ◆ **Expression Parsing:** Compilers and calculators
- ◆ **Depth-First Search:** Graph traversal algorithms
- ◆ **Backtracking Algorithms:** Maze solving, puzzle solving
- ◆ **Syntax Checking:** Matching brackets, tags in code
- ◆ **Memory Management:** Stack memory allocation in operating systems

Best Practices

1. **Choose the right implementation:** Use Python lists for most cases; use linked lists when you need guaranteed dynamic behavior
2. **Handle empty stack cases:** Always check before pop/peek to avoid errors
3. **Use descriptive names:** `push` / `pop` are conventional and clear
4. **Consider thread safety:** Use locks if accessing stack from multiple threads
5. **Document your code:** Especially important for custom implementations

Performance Considerations

- All basic stack operations should be $O(1)$ (constant time)
 - Space complexity is $O(n)$ where n is the number of elements
 - For Python lists: amortized $O(1)$ for push due to dynamic resizing
 - For linked lists: guaranteed $O(1)$ for all operations
-

Conclusion

Stacks are a fundamental data structure that every programmer should master. Whether you're implementing undo functionality, parsing expressions, or traversing graphs, understanding stacks will make you a more effective developer. Python's flexibility allows you to choose between simple list-based implementations for quick solutions or more sophisticated linked list implementations for specific performance requirements.

Key Takeaway: Start with Python's list-based stack for simplicity and performance. Only move to linked lists when you have specific requirements that justify the added complexity.